

Name : Soham Bhattacharya

ID : B2430059

BDA 24-46

Lab 03 Assignment

✓ 3 Random MNIST Images with Labels

✓ Generating 3 random indices

```
import random as rand
x1 = rand.randint(0,60000)
x2 = rand.randint(0,60000)
x3 = rand.randint(0,60000)
```

```
new_x = [x1, x2, x3]
```

```
new_x
```

```
↔ [12355, 33191, 59025]
```

```
import matplotlib.pyplot as plt
from keras.datasets import mnist

# Load the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
X_train.shape
```

```
↔ (60000, 28, 28)
```

```
X_train[59999]
```

```
↔ ndarray (28, 28) show data
```



✓ Printing 3 digits placed at those randomly generated indices in the same plot

```
# Set up the figure for subplots
fig, axes = plt.subplots(1, 3, figsize=(10, 4))
```

```

j=0
# Loop to display three different images
for i in new_x:
    axes[j].imshow(X_train[i], cmap='gray') # Show the image in grayscale
    axes[j].set_title(f"Label: {y_train[i]}") # Display the label
    axes[j].axis('off') # Turn off the axis
    j+=1

plt.show()

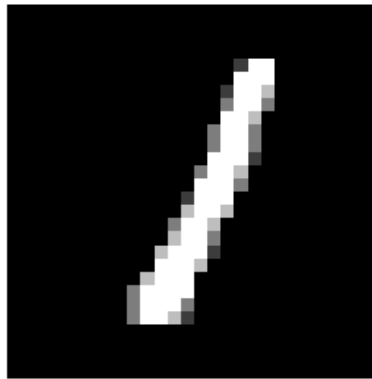
```



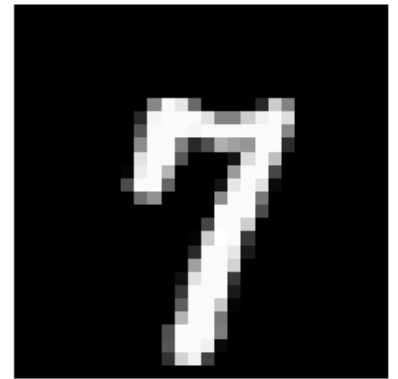
Label: 3



Label: 1



Label: 7



✓ Individually plotting the digits

```

import matplotlib.pyplot as plt

def plot_digit(image_data):
    image = image_data.reshape(28, 28)
    plt.imshow(image, cmap="binary")
    plt.axis("off")

```

```

# Individual Plots

# First entry
some_digit = X[x1]
plot_digit(some_digit)

plt.show()

# Second entry
some_digit = X[x2]
plot_digit(some_digit)

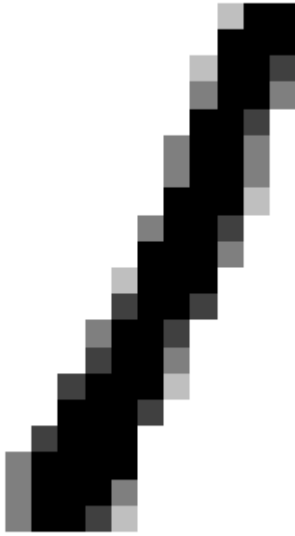
plt.show()

# Third entry
some_digit = X[x3]

```

```
plot_digit(some_digit)
```

```
plt.show()
```



✓ Binary Classification : Class 8 vs Rest

```
from sklearn.datasets import fetch_openml

mnist = fetch_openml('mnist_784', as_frame=False)
```

```
X, y = mnist.data, mnist.target
X
```

```
⇒ array([[0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        ...,
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0],
        [0, 0, 0, ..., 0, 0, 0]])
```

```
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

✓ Training a Binary Classifier

```
y_train_8 = (y_train == '8') # True for all 8s, False for all other digits
y_test_8 = (y_test == '8')
```

```
from sklearn.linear_model import SGDClassifier
```

```
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_8)
```

```
⇒  SGDClassifier ⓘ ?
SGDClassifier(random_state=42)
```

```
some_digit = X[x1]
# print(f'Some Digit = {some_digit}')
sgd_clf.predict([some_digit])
```

```
⇒ array([False])
```

✓ Classification Score

✓ Measuring Accuracy Using Cross-Validation

```
from sklearn.model_selection import cross_val_score

cross_val_score(sgd_clf, X_train, y_train_8, cv=3, scoring="accuracy")

↵ array([0.909 , 0.9286 , 0.88285])
```

✓ Confusion Matrix

```
from sklearn.model_selection import cross_val_predict

y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_8, cv=3)
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_train_8, y_train_pred)
cm
```

```
↵ array([[50006,  4143],
        [ 1448,  4403]])
```

```
y_train_perfect_predictions = y_train_8 # pretend we reached perfection
confusion_matrix(y_train_8, y_train_perfect_predictions)
```

```
↵ array([[54149,    0],
        [    0,  5851]])
```

✓ Precision

```
from sklearn.metrics import precision_score, recall_score

precision_score(y_train_8, y_train_pred)
```

```
↵ 0.515211794991809
```

```
# extra code – this cell also computes the precision: TP / (FP + TP)
cm[1, 1] / (cm[0, 1] + cm[1, 1])
```

```
↵ 0.515211794991809
```

✓ Recall

```
recall_score(y_train_8, y_train_pred)
```

```
↵ 0.7525209365920356
```

```
# extra code – this cell also computes the recall: TP / (FN + TP)
cm[1, 1] / (cm[1, 0] + cm[1, 1])
```

↩ 0.7525209365920356

✓ F1-Score

```
from sklearn.metrics import f1_score

f1_score(y_train_8, y_train_pred)
```

↩ 0.6116552059456831

```
# extra code – this cell also computes the f1 score
cm[1, 1] / (cm[1, 1] + (cm[1, 0] + cm[0, 1]) / 2)
```

↩ 0.6116552059456831

✓ ROC-AUC and Trade-offs

✓ Precision/Recall Trade-off

```
y_scores = sgd_clf.decision_function([some_digit])
y_scores
```

↩ array([-7999.82404651])

```
threshold = 0
y_some_digit_pred = (y_scores > threshold)
```

```
y_some_digit_pred
```

↩ array([False])

```
# extra code – just shows that y_scores > 0 produces the same result as
# calling predict()
y_scores > 0
```

↩ array([False])

```
threshold = 3000
y_some_digit_pred = (y_scores > threshold)
y_some_digit_pred
```

```
➡ array([False])
```

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_8, cv=3, method="decision_
```

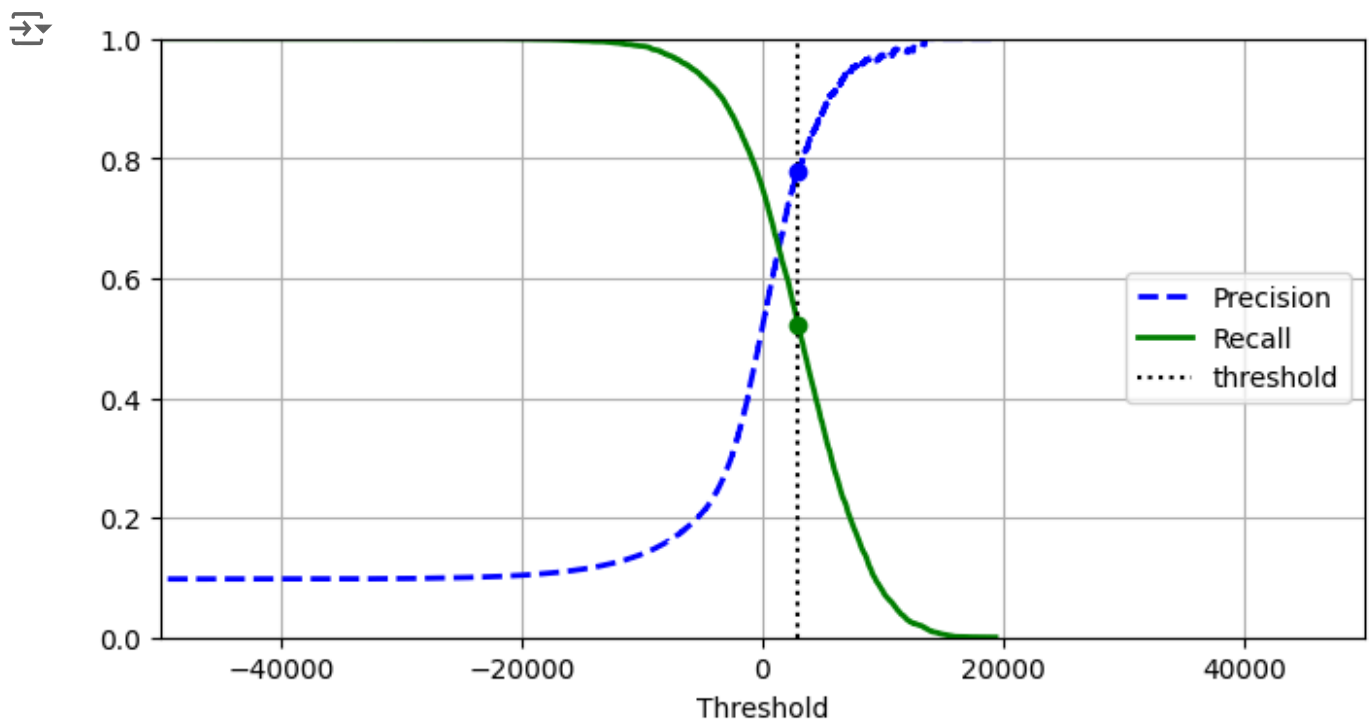
```
from sklearn.metrics import precision_recall_curve
```

```
precisions, recalls, thresholds = precision_recall_curve(y_train_8, y_scores)
```

```
plt.figure(figsize=(8, 4)) # extra code – it's not needed, just formatting
plt.plot(thresholds, precisions[:-1], "b--", label="Precision", linewidth=2)
plt.plot(thresholds, recalls[:-1], "g-", label="Recall", linewidth=2)
plt.vlines(threshold, 0, 1.0, "k", "dotted", label="threshold")
```

```
# extra code – this section just beautifies and saves Figure 3–5
idx = (thresholds >= threshold).argmax() # first index ≥ threshold
plt.plot(thresholds[idx], precisions[idx], "bo")
plt.plot(thresholds[idx], recalls[idx], "go")
plt.axis([-50000, 50000, 0, 1])
plt.grid()
plt.xlabel("Threshold")
plt.legend(loc="center right")
# save_fig("precision_recall_vs_threshold_plot")

plt.show()
```



```
import matplotlib.patches as patches # extra code – for the curved arrow
```

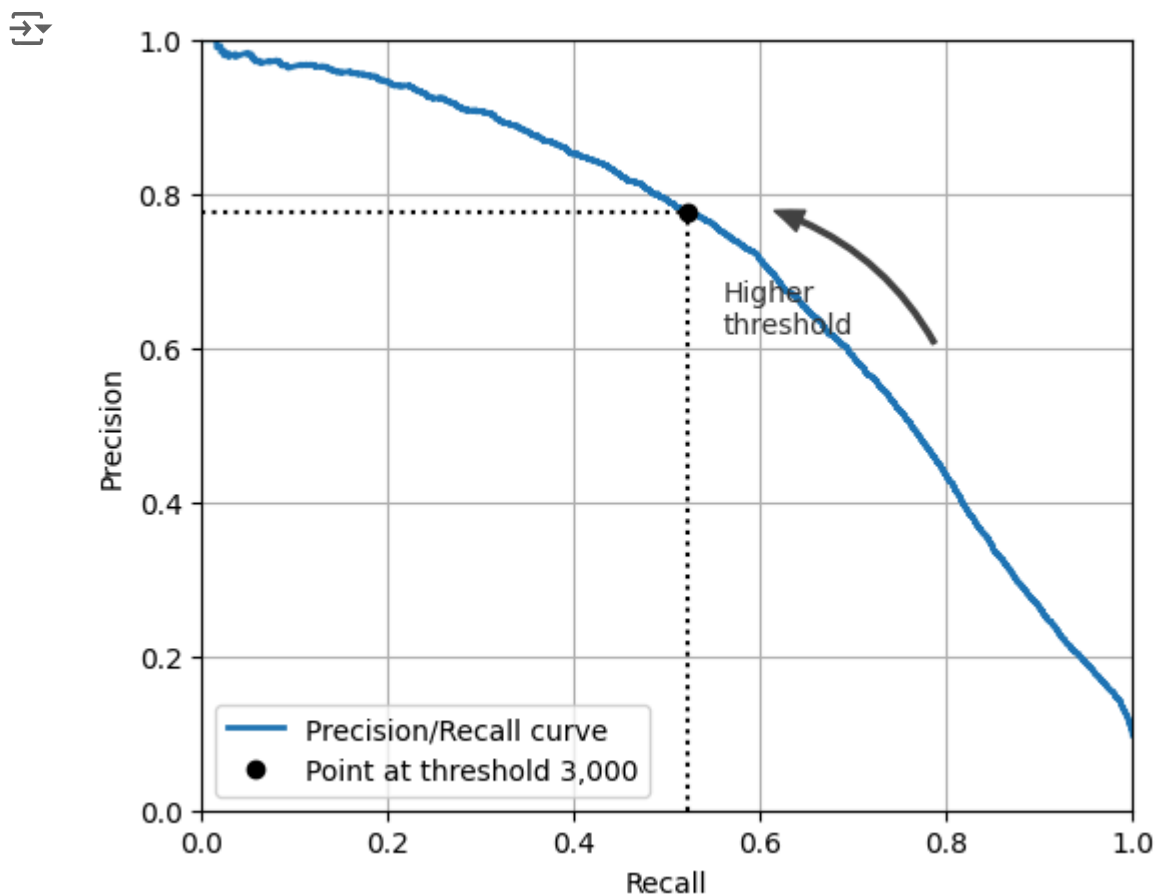
```
plt.figure(figsize=(6, 5)) # extra code – not needed, just formatting
```

```
plt.plot(recalls, precisions, linewidth=2, label="Precision/Recall curve")
```



```
# extra code – just beautifies and saves Figure 3–6
plt.plot([recalls[idx], recalls[idx]], [0., precisions[idx]], "k:")
plt.plot([0.0, recalls[idx]], [precisions[idx], precisions[idx]], "k:")
plt.plot([recalls[idx]], [precisions[idx]], "ko",
         label="Point at threshold 3,000")
plt.gca().add_patch(patches.FancyArrowPatch(
    (0.79, 0.60), (0.61, 0.78),
    connectionstyle="arc3,rad=.2",
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",
    color="#444444"))
plt.text(0.56, 0.62, "Higher\nthreshold", color="#333333")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.axis([0, 1, 0, 1])
plt.grid()
plt.legend(loc="lower left")
# save_fig("precision_vs_recall_plot")

plt.show()
```



```
idx_for_90_precision = (precisions >= 0.90).argmax()
threshold_for_90_precision = thresholds[idx_for_90_precision]
threshold_for_90_precision
```

```
5558.688951079276
```

```
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
```

```
precision_score(y_train_8, y_train_pred_90)
```

```
⇒ 0.9003419638495359
```

```
recall_at_90_precision = recall_score(y_train_8, y_train_pred_90)  
recall_at_90_precision
```

```
⇒ 0.3149888907878995
```

✓ The ROC Curve

```
from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_train_8, y_scores)
```

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()  
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]
```

```
plt.figure(figsize=(6, 5)) # extra code – not needed, just formatting  
plt.plot(fpr, tpr, linewidth=2, label="ROC curve")  
plt.plot([0, 1], [0, 1], 'k:', label="Random classifier's ROC curve")  
plt.plot([fpr_90], [tpr_90], "ko", label="Threshold for 90% precision")
```

```
# extra code – just beautifies and saves Figure 3–7  
plt.gca().add_patch(patches.FancyArrowPatch(  
    (0.20, 0.89), (0.07, 0.70),  
    connectionstyle="arc3,rad=.4",  
    arrowstyle="Simple, tail_width=1.5, head_width=8, head_length=10",  
    color="#444444"))
```

```
plt.text(0.12, 0.71, "Higher\nthreshold", color="#333333")  
plt.xlabel('False Positive Rate (Fall-Out)')  
plt.ylabel('True Positive Rate (Recall)')  
plt.grid()  
plt.axis([0, 1, 0, 1])  
plt.legend(loc="lower right", fontsize=13)  
# save_fig("roc_curve_plot")
```

```
plt.show()
```